Kaleido

Presenter's companion

Nine-slide executive overview | v5 | 16 October 2026

Make the step concept tangible. Show where AI guidance comes from. Explain the delivery advantage.

Timing

Plan for about 19 minutes, including pauses and diagram walkthroughs. Use the 15-minute route below for a tighter slot; the extended route is about 23.5 minutes, leaving roughly 1.5 minutes for questions in a 25-minute slot.

One-page cue sheet

#	Slide focus	Remember	Std / Fast
1	What it is	A .NET query and process framework, not just a catalog.	1:00 / 0:45
2	Why this project	Make steps visible; remove duplicated workflow ownership.	2:00 / 1:30
3	The core model	Information, actions, discovery; a step is not a screen.	2:00 / 1:30
4	The code makes it click	The service declares it; the handler does it; Kaleido runs it.	3:00 / 2:30
5	The MCP guidance loop	Connect, receive, call, continue - details arrive as needed.	2:30 / 2:00
6	Change + add products	Change the service, not an agent copy. Add products via discovery.	2:30 / 2:00
7	The Devin session	An unlisted required step arrived in next and was called.	3:00 / 2:30
8	Fit + authority	The service and host keep business meaning and authority.	2:00 / 1:30
9	The close	Connect without teaching. Evolve without duplicating. Expand.	1:00 / 0:45

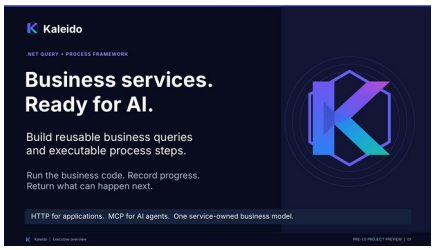
Ready answer: "What is Kaleido?"

Kaleido is a .NET framework for building reusable business queries and executable steps. It runs the business code, records process progress, and supplies a compatible AI agent with the tools and guidance needed to continue - without a separately maintained product workflow inside the agent.

Use the full Say sections to rehearse. Present from Remember and On screen. The reminders and source notes are reference material, not lines to read aloud.

SLIDE 01 / 1:00 STANDARD

Business services. Ready for AI.



REMEMBER

A .NET framework that runs business actions and supplies the information needed to continue.

On screen: Open with the definition, not the technology diagram. Pause after 'the service remains the source of truth.'

Say

Kaleido is a .NET framework for building business services as reusable queries and executable steps. It runs the business code, records process progress, and returns what can happen next.

A query might find a member. A step might capture that member against a request. Applications can use those capabilities through HTTP. A compatible AI agent can connect through MCP, the Model Context Protocol, and receive the product-specific tools and guidance from Kaleido.

The important point is that we are not putting a second business workflow inside the agent. The service remains the source of truth. I will show why I started the project, where that guidance comes from, and why it changes how teams can deliver conversational AI. This is a pre-1.0 project preview.

Transition

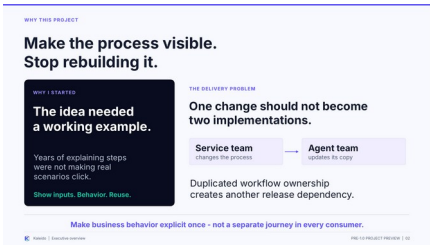
First, why build this rather than explain the idea one more time?

Presenter reminder (not spoken): AI-native means the implemented MCP interface, not a built-in model or release certification.

Source basis: Working notes 01-03; v4 slides 01-02; cleanup-chatgpt.md section 3.

SLIDE 02 / 2:00 STANDARD

Make the process visible. Stop rebuilding it.



REMEMBER

A working example made the step concept tangible; a shared service model addresses duplicated process ownership.

On screen: Start with the personal story on the left. Then trace the service-team and agent-team dependency on the right.

Say

For several years, I had been trying to explain the step concept. I could describe it, but when the discussion moved into real scenarios, the explanation was not making it clear how the pieces would work together.

What I wanted people to see was the behavior: what an action needs, when it can run, what changes after it completes, and what happens when more information is needed. Kaleido began as a working implementation we could run, inspect, and discuss, rather than another abstract explanation.

There is also a delivery problem that this explicit model addresses. When the service owns one version of the process and the agent owns another, a business change becomes a two-team project. The product team can finish the service and still be waiting for the conversational-AI team to update its workflow, prompts, mappings, testing, and release.

The goal is not to remove the agent team or eliminate testing. It is to avoid making that team maintain a duplicate product workflow. The product team owns the business behavior and publishes the information needed to interact with it. The same investment can support a website, an integration, or an AI assistant without redefining the action for each one.

AI was not the original reason I started the project. It has become an important use of the same explicit model.

Transition

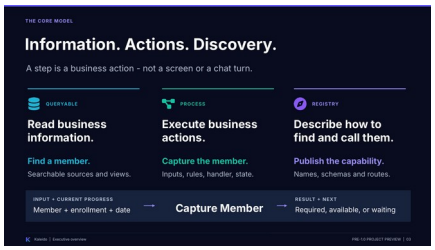
To make that concrete, there are only three framework concepts we need for this overview.

Presenter reminder (not spoken): Origin is your account, paraphrased. Delivery savings are a proposition, not a measured result.

Source basis: Your origin and delivery-dependency accounts; working notes 02 and 06; v4 slides 03-04.

SLIDE 03 / 2:00 STANDARD

Information. Actions. Discovery.



REMEMBER

Queryable reads; Process acts; Registry describes. A step is a business action, not a screen.

On screen: Move across the three columns. Then point to Capture Member and the result-plus-next line.

Say

Queryable is the information side: the sources and views the business exposes for search and retrieval. In our example, that is how we find a member and the associated enrollment information.

Process is the action side. Capture Member is a named action with inputs, process relationships, and a business handler that performs the work. Registry is the catalog describing those capabilities so consumers can find them and understand how to call them.

A step is not a screen, a button, or a chat turn. A web page may help a person select a member. An AI assistant may help the person find the same record in conversation. Those are different interactions around the same business action.

The action runs against current process progress. The result is more than success or failure: it also describes how work may continue. There may be a particular required step, several available choices, or specific questions that need answers.

That does not mean every step is interchangeable or that the model decides the order. Declared relationships, recorded progress, and direction from the business handler constrain what is permitted. The person supplies choices and facts; the application retains the business meaning. We are making that behavior explicit enough for different consumers to use.

Transition

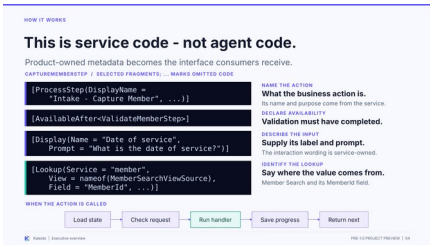
The natural question is: where do those descriptions and rules actually come from? Here is a real piece of the service.

Presenter reminder (not spoken): Required and available are different; the diagram is not an exhaustive state list.

Source basis: v4 slides 02 and 05; cleanup-chatgpt.md section 3; supplied CaptureMemberStep.

SLIDE 04 / 3:00 STANDARD

This is service code - not agent code.



REMEMBER

The product team declares the interaction contract. The handler does the work; Kaleido runs the process around it.

On screen: Highlight ProcessStep, AvailableAfter, Display, and Lookup in that order. Spend 60-90 seconds on the code, then trace the runtime strip.

Say

This is an excerpt from CaptureMemberStep, not the full class. You do not need to read the C# syntax in detail. I want to point out four places where the product team describes the action.

ProcessStep names the business action and carries its description. AvailableAfter says member validation must have completed before this action can become available. That is an availability condition; it does not automatically execute the step or, by itself, make it the required next action.

Display provides a readable label and prompt for an input. In the full declaration there are also descriptions explaining what the inputs mean. Lookup identifies the business query and result field that supply a value. Here it points to member search rather than expecting a person or model to invent an internal member identifier.

That information is owned by the product service. Once the declarations are registered and the adapters configured, Kaleido publishes the model to consumers. The AI receives tool descriptions, input schemas, and guidance through MCP. It is not expected to read this source file, and we do not separately code a Capture Member workflow inside Devin.

The declaration is not the implementation. The associated business handler still performs the action and checks its business meaning. The framework loads the process state, checks the request and process rules, invokes the handler, saves progress, and returns the result and next work. That is the strip along the bottom.

So the important separation is: the product team describes and implements the business action; Kaleido supplies the shared execution and discovery mechanisms. The same declaration supports a callable action without a second, agent-specific definition of that action.

Transition

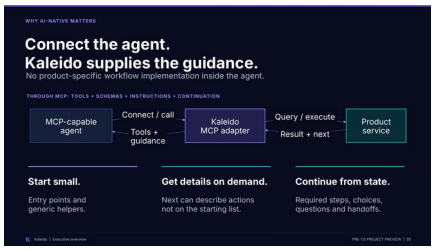
Now follow the information from that service definition to a generic AI agent.

Presenter reminder (not spoken): Fragments are not a complete class. AvailableAfter does not automatically execute a step. Lookup wording is not proof of business validation.

Source basis: Supplied CaptureMemberStep and working notes 07; cleanup-chatgpt.md section 3.

SLIDE 05 / 2:30 STANDARD

Connect the agent. Kaleido supplies the guidance.



REMEMBER

No product-specific workflow in the agent. Tools and continuation details arrive through MCP, including on demand.

On screen: Trace the outbound calls across the diagram, then the return arrows. Use the three short callouts as the meaning of that loop.

Say

The agent needs the generic ability to connect to MCP and use tools. It does not need a separately implemented model of this product's workflow before it connects.

The adapter derives the agent-facing tools and input schemas from the caller-visible registry. The product supplies the descriptions and lookup relationships. The runtime supplies process state, outcomes, and continuation information. The model uses that information as it interacts with the person.

The information does not all have to arrive in one enormous tool list. In the FrontDoor mode shown in my Devin session, the agent begins with entry points and generic helpers. Later responses can describe additional capabilities when the process needs them, including what arguments they accept and how to invoke them.

Follow the arrows. The agent calls a query or step through the adapter. The business service executes the request and returns the result and current next work. The adapter makes that usable to the agent, including required actions, available choices, lookup details, or questions to ask the person. The consumer continues from the returned guidance.

The agent still interprets the person's intent and uses its general tool-use ability. What we avoid is teaching and maintaining a second product-specific step sequence inside it. And the process state lives in the service, not only in the chat transcript.

That is what AI-native means in this presentation: an interface that supplies the operational information an agent needs to use the business, not a language model embedded inside the framework.

Transition

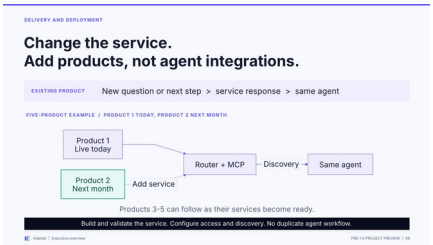
Once that workflow no longer has to be duplicated in the agent, the delivery model changes too.

Presenter reminder (not spoken): A generic compatible client must consume refreshed discovery and current responses. Guidance is not proof of permission or consent.

Source basis: Working notes 01 and 05; session sections 0, 2-3; cleanup-chatgpt.md sections 3 and 15.

SLIDE 06 / 2:30 STANDARD

Change the service. Add products, not agent integrations.



REMEMBER

Remove duplicate workflow ownership, not the product implementation or the need to validate it.

On screen: Start with an existing-process change. Then use Product 1 today and Product 2 next month to explain expansion through the same connection.

Say

There are two different benefits here. The first is changing a product we already expose. Suppose the service needs an additional question or a different required next step. When that change is expressed through the supported contract, the updated service supplies the new question or continuation. We do not need a second implementation change just to teach the agent the new workflow.

That removes a particular delivery dependency. The service team is not automatically waiting for the team that owns conversational AI to modify and release its matching process definition. The service change still needs testing, but duplicated agent development is not inherently part of the work.

The second benefit is adding a product. Imagine a company with five products that starts by bringing only Product 1 onto conversational AI. Next month, it wants to add Product 2. The team builds the new Kaleido service, configures the router and MCP adapter to reach and expose it, configures access, and validates the integration.

The existing agent receives the new capabilities through discovery. It does not require a newly authored Product 2 workflow, tool mapping, or prompt sequence. The same agent and MCP connection now have access to another product, within the contracts the integration already supports. The other three products can follow as their services are ready.

This is not zero work. The product team still builds the product, and deployment and access have owners. What disappears is the need to reproduce that product's workflow inside the conversational agent. The agent finds out through discovery, not through a development ticket.

Transition

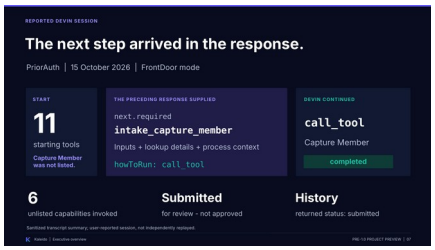
Here is a session I ran that makes the runtime-guidance mechanism visible.

Presenter reminder (not spoken): Supported-contract changes still require service work, routing/access configuration, refresh, and validation. The five-product example is illustrative.

Source basis: Your service-change and five-product examples; working notes 02-03; registry/MCP paths in cleanup-chatgpt.md section 3.

SLIDE 07 / 3:00 STANDARD

The next step arrived in the response.



REMEMBER

Capture Member was not initially listed. The previous response supplied its details, and Devin used call_tool to run it.

On screen: Use the three beats from left to right. Pause on 'not on the starting list.' End on submission plus the returned history status, not on clinical details.

Say

This is based on the PriorAuth session I ran with Devin on October 15. The slide summarizes the transcript and leaves out the member, provider, and process identifiers. We are not replaying a live demo here.

The starting tool list had eleven tools: entry points, searches, and helpers. It did not enumerate all the downstream actions. In particular, Capture Member was not on that starting list.

After the agent called Validate Member, the response named Capture Member as the required next step. It supplied the input details, lookup information, process context, and how to run the action through call_tool. Devin then used that information to invoke the action, and the result was completed.

That is the same named business action whose declaration we just examined. The code example shows where the service-owned descriptions begin; the session shows an agent receiving the instructions needed to continue.

The interaction went on through the Radiology handoff, provider selection, and service-supplied question sets. Before one provider action, the agent explained that the choice would close earlier member and service changes. That is more informative than just returning the name of another function.

Across the supplied session, six unlisted capabilities were invoked through call_tool. After review and the user's attestation answer, the service reported submission. A final history query returned a submitted record. The outcome was submitted for review, not approved.

The takeaway is not merely that an assistant had a convincing conversation. It used details supplied in the preceding service responses to perform the work and obtain a recorded outcome. That is the architectural point I want you to see.

Transition

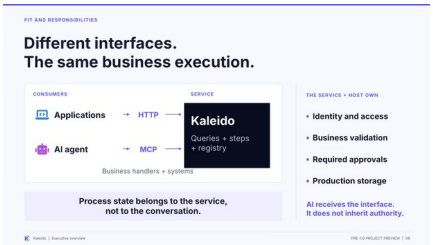
The AI interaction is useful, but the business service still owns execution and authority.

Presenter reminder (not spoken): Reported session, not independently replayed. The named step matches the example, not verified source bytes. Submission is not approval.

Source basis: Devin session sections 0, 2-8 and Summary; working notes 04-05; supplied step declaration.

SLIDE 08 / 2:00 STANDARD

Different interfaces. The same business execution.



REMEMBER

Applications and AI use the same capabilities; the service and host retain business authority.

On screen: Trace the two consumer lanes into the service. Then use the responsibility panel to distinguish guidance from enforcement.

Say

The shared business model is useful beyond the AI channel. Applications can call it through HTTP; the AI agent uses the MCP adapter. The action and its implementation remain in the business service.

That means an existing workflow or case-management platform can keep assignments, human tasks, and service-level management. Web, voice, and assisted channels keep their presentation responsibilities. External standards and partner adapters keep their exchange boundaries. These are places the architecture can fit, not a claim that Kaleido ships every connector.

The process has its own identity and current state. With the appropriate durable storage and access controls, a later authorized consumer can read that state rather than reconstructing the business process from a chat transcript. That is an architectural pattern, not proof that every channel transfer has been tested.

The authority boundary is equally important. The host establishes identity and access. The application owns business validation, record-level access, approvals, and the meaning of the action. Tool descriptions and instructions help the interaction; they do not independently prove permission or human consent.

This remains a pre-1.0 preview. The earlier reviews identify hardening work around credentials, retries and completion, and real-client behavior. We are not treating the successful session as closure of those concerns. The story is an implemented framework and a reusable interface, with production responsibilities kept explicit.

Transition

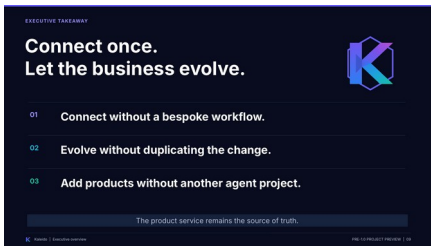
That brings the original idea and the AI delivery benefit together.

Presenter reminder (not spoken): No bundled vendor connectors or completed release guarantees are implied. Process identity is not case-level authorization.

Source basis: v4 slides 08 and 11-13; cleanup-chatgpt.md sections 3, 5 and 15; cleanup-devin.md KAL-REL-001 and KAL-SEC-001.

SLIDE 09 / 1:00 STANDARD

Connect once. Let the business evolve.



REMEMBER

Connect without a bespoke workflow. Evolve without duplicating the change. Add products without another agent project.

On screen: Read the three takeaways slowly. Finish, pause, and invite questions rather than adding more architecture detail.

Say

I started Kaleido to make the step concept something people could see working. The same explicit business model now gives a compatible AI agent the tools and guidance it needs without a second product workflow being maintained inside the agent.

There are three things I want you to remember. Connect without teaching the agent your product's step sequence. Evolve the workflow without duplicating the same change in an agent implementation. Add products through the existing router and MCP integration rather than creating another agent-development project.

The product service remains the source of truth for the business action, current process state, and how the work may continue. The interface can change, and the product portfolio can grow, without moving that ownership into the conversation.

Transition

Pause, then invite questions about one concrete product process or adoption path.

Presenter reminder (not spoken): Keep the supported-contract and pre-1.0 scope. Do not turn independence into a no-testing or universal-compatibility claim.

Source basis: Working notes central message, 01-03 and 06-07; your origin and rollout accounts.

Q&A / SUGGESTED ANSWERS

Questions you are likely to get

The step model and AI interface

What does the developer still have to build?

The queries, step declarations, and business handlers, plus the connections to the real business systems. The host still needs identity, access, storage, and deployment configuration. Kaleido supplies the common query/process runtime, registry, and adapters around that work; it does not automatically turn arbitrary existing APIs into Kaleido steps.

Basis: Slides 03-04; supplied step; v4 build-time explanation.

Is the code shown a prompt for the AI?

No. It is service-owned C# metadata and a step contract. The MCP adapter derives a tool-facing interface from the registry. The agent receives tool and input information, not this source file. Display prompts describe an input; they are not the same thing as runtime information-request questions.

Basis: Slide 04; supplied CaptureMemberStep; working notes 07.

Does AvailableAfter mean this action must run next?

No. It establishes an availability condition after the named step completes. Required-next direction comes from the running process and can be returned by the business handler. In the reported Devin session, Capture Member was returned as required; that is not something the attribute alone proves.

Basis: Slides 03-04 and 07; session section 2; working notes 07.

Can the agent act without any understanding or configuration?

It needs generic MCP and tool-use support, an authorized connection, and the ability to use current tool descriptions and responses. It still interprets user intent. The point is that it does not need a separately implemented, product-specific workflow before connecting. Guidance does not guarantee that every model or client follows it correctly.

Basis: Slide 05; working notes 01 and qualifications.

Return to the central idea: the product service owns the process; the agent receives the information needed to use it.

Q&A / SUGGESTED ANSWERS

Questions you are likely to get

Delivery, evidence, and readiness

Does a service change really mean no agent work or testing?

For workflow changes expressed through the existing supported contracts, a matching product-specific agent implementation is not inherently required. Service testing, registry refresh, access changes, and end-to-end validation still matter. A new client capability, breaking contract, or hard-coded consumer behavior can still require coordination.

Basis: Slide 06; working notes 02-03 and qualifications.

What does the Devin example prove about adding another product?

It reports successful use of runtime-supplied details, including a required action absent from the starting tool list, and a submitted result returned by the services. It supports the explanation of the mechanism. It did not test adding a second product with an unchanged agent, and it does not establish Devin's complete prior context.

Basis: Slide 07; supplied session; working notes 04-05.

Does this replace our workflow platform, or let the model approve requests?

Neither is the positioning. Kaleido includes a process runtime, but the wider platform can keep assignments, human tasks, and service-level management. The business application retains domain decisions and approval rules. In the supplied session, submitted for review is not approved. Tool hints are not permission or proof of consent.

Basis: Slides 03 and 08; session sections 7-8; v4 platform and authority material.

Is it ready for a production 1.0 release?

This deck is a pre-1.0 preview, not a release approval. The earlier reviews identify credential-delegation, retry/completion, packaging, and other validation work. The successful session is useful evidence of one interaction, not closure of those concerns. Confirm their status against the actual release candidate before making a production-readiness claim.

Basis: Slide 08; cleanup-chatgpt.md sections 5 and 15; cleanup-devin.md KAL-REL-001 and release checklist.

Return to the central idea: the product service owns the process; the agent receives the information needed to use it.